Lab: Nonlinear Least Squares for Modeling Materials

Nonlinear least squares (NLLS) is a widely-used method for modeling data. In NLLS, we wish to fit a model of the form,

```
yhat = g(x,w)
```

where `w` is a vector of paramters and `x` is the vector of predictors. We find `w` by minimizing a least-squares function

```
f(w) = \sum_i (y_i - g(x_i,w))^2
```

where the summation is over training samples `(x_i,y_i)`. This is similar to linear least-squares, but the function `g(x,w)` may not be linear in `w`. In general, this optimization has no closed-form expression. So numerical optimization must be used.

In this lab, we will implement gradient descent on NLLS in a problem of physical modeling of materials. Specifically, we will estimate parameters for expansion of copper as a function of temperature using a real dataset. In doing this lab, you will learn to:

- Set up a nonlinear least squares as an unconstrained optimization function
- Compute initial parameter estimates for a simple rational model
- Compute the gradients of the least squares objective
- Implement gradient descent for minimizing the objective
- Implement momentum gradient descent
- Visualize the convergence of the algorithm

We first import some key packages.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import Ridge, LinearRegression
```

Load the Data

The NIST agency has an excellent [nonlinear regression website] that has several datasets for nonlinear regression problems. In this lab, we will use the data from a NIST study involving the thermal expansion of copper. The response variable is the coefficient of thermal expansion, and the predictor variable is temperature in degrees kelvin.

> Hahn, T., NIST (1979), Copper Thermal Expansion Study. (unpublished}

You can download the data as follows.

```
import urllib.request

url = 'https://itl.nist.gov/div898/strd/nls/data/LINKS/DATA/Hahn1.dat'
req = urllib.request.Request(url, headers={'User-Agent': 'Mozilla/5.0'})
with urllib.request.urlopen(req) as response:
    df = pd.read_csv(response, skiprows=60, sep=' ', skipinitialspace=True, names=['y0','x0

df.head()
```

	y0	x0	dummy
0	0.591	24.41	NaN
1	1.547	34.82	NaN
2	2.902	44.09	NaN
3	2.894	45.07	NaN
4	4.703	54.98	NaN

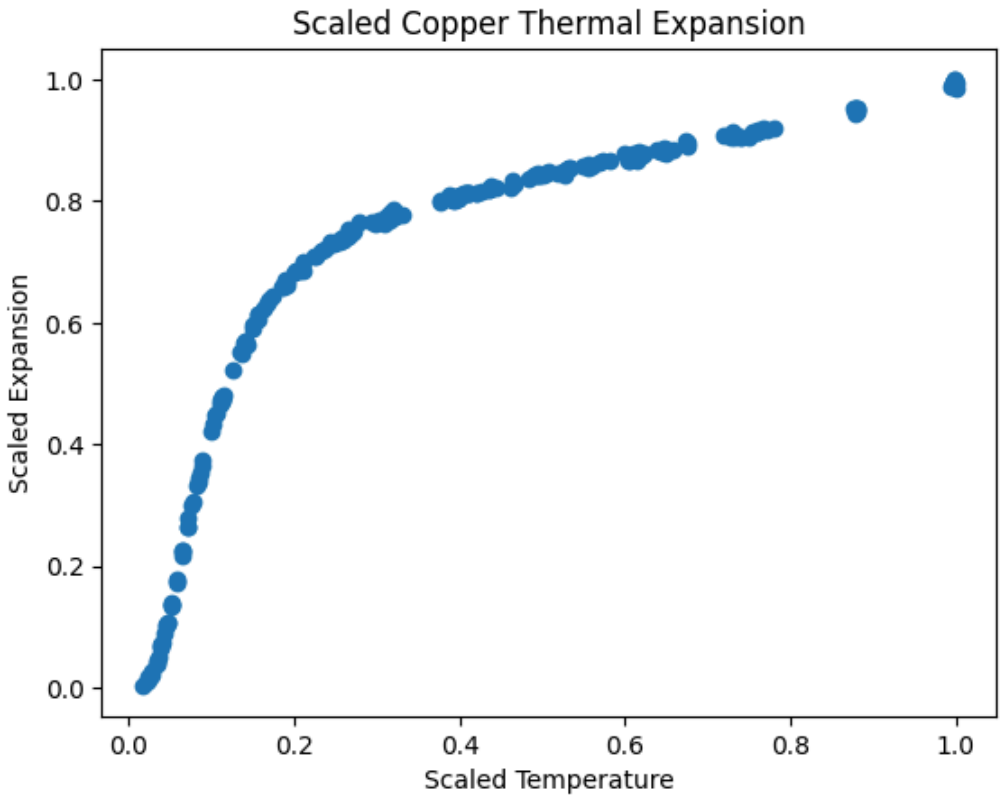
Next steps: Generate code with `df` New interactive sheet

Extract the `x0` and `y0` into arrays. Rescale, `x0` and `y0` to values between `0` and `1` by dividing `x0` and `y0` by the maximum value. Store the scaled values in vectors `x` and `y`. The rescaling will help with the conditioning of the fitting. Plot, `y` vs. `x`.

```
# TODO 1
# x0 = ...
# y0 = ...
# x = x0/np.max(x0)
# y = y0/np.max(y0)
# plt.plot(...)

x0 = df['x0'].values
y0 = df['y0'].values
x = x0/np.max(x0)
y = y0/np.max(y0)
plt.plot(x, y, 'o')
plt.xlabel('Scaled Temperature')
plt.ylabel('Scaled Expansion')
plt.title('Scaled Copper Thermal Expansion')
```

```
Text(0.5, 1.0, 'Scaled Copper Thermal Expansion')
```

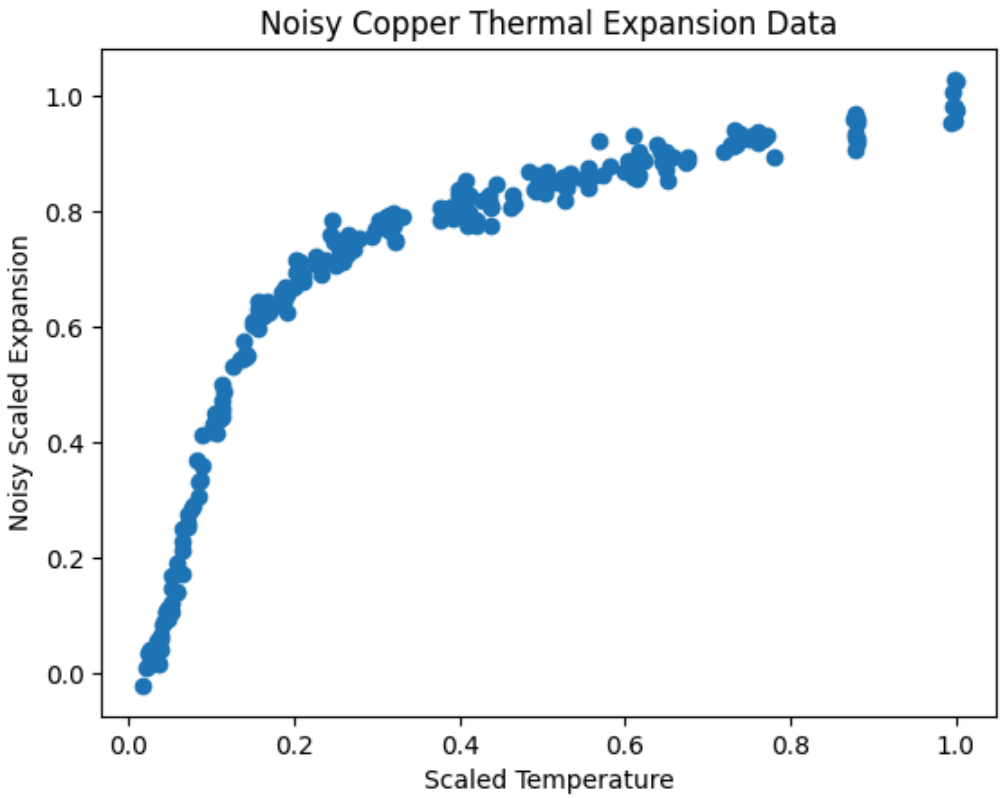


To make the problem a little more challenging, we will add some noise. Add random Gaussian noise with mean 0 and std. dev = 0.02 to `y`. Store the noisy results in `yn`. You can use the `np.random.normal()` function to add Gaussian noise. Plot `yn` vs. `x`.

```
# TODO 2
# yn = y + ...
yn = y + np.random.normal(0, 0.02, size=y.shape)

plt.plot(x, yn, 'o')
plt.xlabel('Scaled Temperature')
plt.ylabel('Noisy Scaled Expansion')
plt.title('Noisy Copper Thermal Expansion Data')
```

```
Text(0.5, 1.0, 'Noisy Copper Thermal Expansion Data')
```



Split the data `(x,yn)` into training and test. Let `xtr,ytr` be training data and `xts,yts` be the test data. You can use the `train_test_split` function. Set `test_size=0.33` so that 1/3 of the samples are held out for test.

```
from sklearn.model_selection import train_test_split

# TODO 3
# xtr, xts, ytr, yts = ...
xtr, xts, ytr, yts = train_test_split(x, yn, test_size=0.33, random_state=42)
```

Initial Fit for a Rational Model

The NIST website suggests using a *rational* model of the form,

```
yhat = (a[0] + a[1]*x + ... + a[d]*x^d)/(1 + b[0]*x + ... + b[d-1]*x^d)
```

with `d=3`. The model parameters are `w = [a[0],...,a[d],b[0],...,b[d-1]]` so there are `2d+1` parameters total. Complete the function below that takes vectors `w` and `x` and predicts a set of values `yhat` using the above model.

```
def predict(w,x):

    # Get the length
```

```
    d = (len(w)-1)//2

    # TODO 4.  Extract a and b from w
    # a = ...
    # b = ...
    a = w[:d+1]
    b = w[d+1:]

    # TODO 5.  Compute yhat.  You may use the np.polyval function
    # But, remember you must flip the order the a and b
    # yhat = ...
    numerator = np.polyval(a[::-1], x)
    denominator = 1 + np.polyval(np.append(b[::-1], 0), x)
    yhat = numerator / denominator
    return yhat
```

When we fit with a nonlinear model, most methods only get convergence to a local minima. So, you need a good initial condition. For a rational model, one way to get is to realize that if:

```
y ~= (a[0] + a[1]*x + ... + a[d]*x^d)/(1 + b[0]*x + ... + b[d-1]*x^d)
```

Then:

```
y ~= a[0] + a[1]*x + ... + a[d]*x^d - b[0]*x*y + ... - b[d-1]*x^d*y.
```

So, we can solve for the the parameters `w = [a,b]` from linear regression of the predictors,

```
Z[i,:] = [ x[i], ... , x[i]**d, y[i]*x[i], ... , y[i}*x[i]**d ]
```

```
    d = 3

    # TODO 6.  Create the transformed feature matrix
    # Z = ...
    Z = np.zeros((len(xtr), 2*d))
    for j in range(d):
        Z[:, j] = xtr**(j+1)
        Z[:, d+j] = -ytr * (xtr**(j+1))

    # TODO 7.  Fit with parameters with linear regression
    # regr = LinearRegression()
    # regr.fit(...)
    regr = LinearRegression(fit_intercept=True)
    regr.fit(Z, ytr)

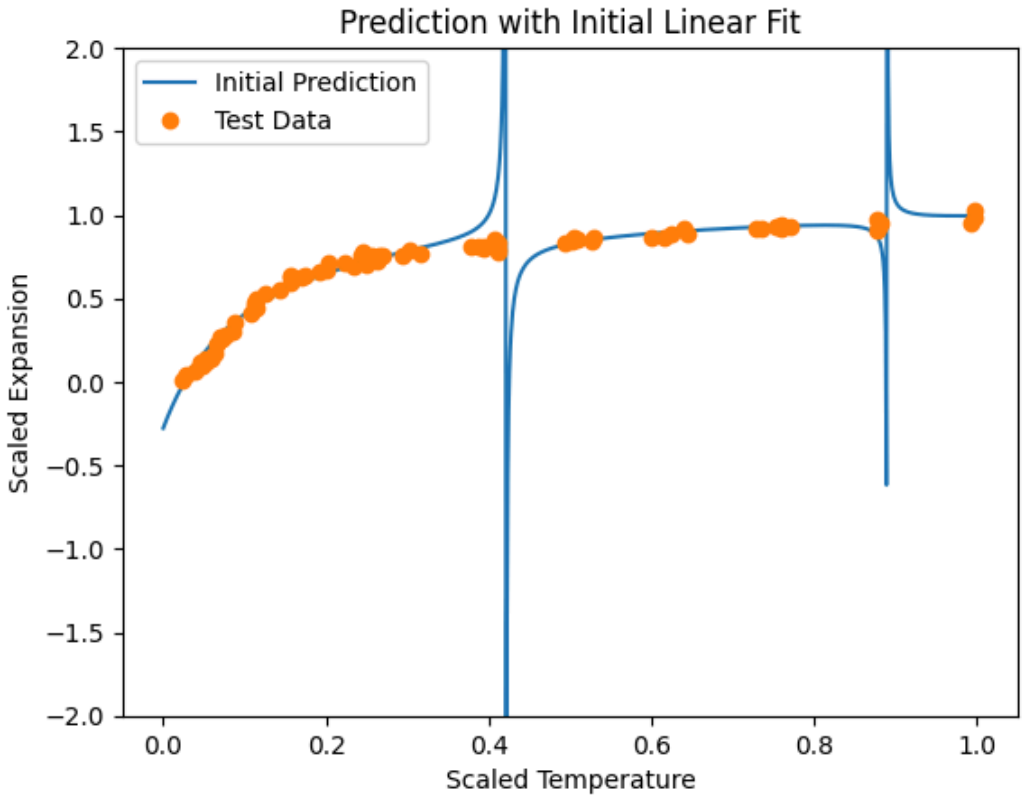
    # TODO 8
    # Extract the parameters from regr.coef_ and regr.intercept_ and store the parameter vect
    # winit = ...
    a = regr.coef_[:d]
    b = regr.coef_[d:]
    winit = np.concatenate(([a0], a, b))
```

Now plot the predicted values of the `yhat` vs. `x` using your estimated parameter `winit` for 1000 values `x` in `[0,1]`. On the same plot, plot `yts` vs. `xts`. You will see that you get a horrible fit.

```
# TODO 9
# xp = ...
# yhat = ...
# plot(...)
xp = np.linspace(0, 1, 1000)
yhat = predict(winit, xp)

plt.plot(xp, yhat, '-', label='Initial Prediction')
plt.plot(xts, yts, 'o', label='Test Data')
plt.xlabel('Scaled Temperature')
plt.ylabel('Scaled Expansion')
plt.title('Prediction with Initial Linear Fit')
plt.legend()
plt.ylim([-2, 2])
```

```
(-2.0, 2.0)
```



The reason the previous fit is poor is that the denominator in `yhat` goes close to zero. To avoid this problem, we can use Ridge regression, to try to keep the parameters close to zero. Re-run the fit above with `Ridge` with `alpha = 1e-3`. You should see you get a reasonable, but not perfect fit.

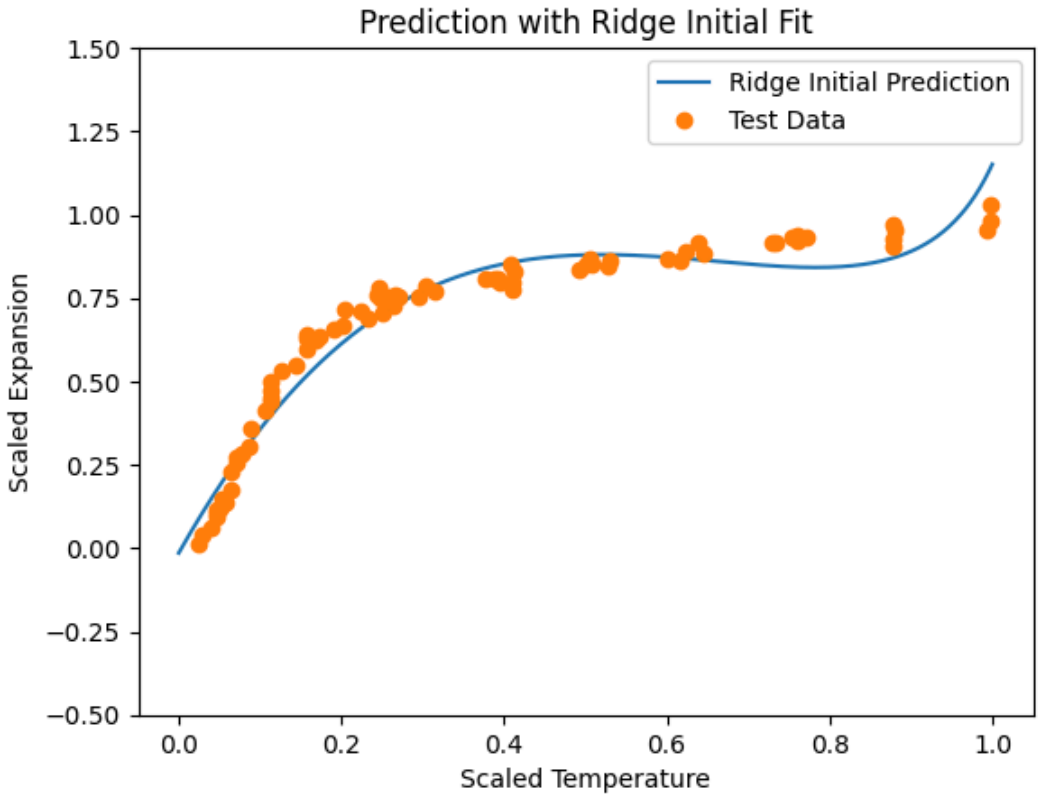
```
# TODO 10.  Fit with parameters with linear regression
# regr = Ridge(alpha=1e-3)
# regr.fit(...)
regr = Ridge(alpha=1e-3, fit_intercept=True)
regr.fit(Z, ytr)
```

```
# TODO 11
# Extract the parameters from regr.coef_ and regr.intercept_
# winit = ...
a0 = regr.intercept_
a = regr.coef_[:d]
b = regr.coef_[d:]
winit = np.concatenate(([a0], a, b))

# TODO 12
# Plot the results as above.
xp = np.linspace(0, 1, 1000)
yhat = predict(winit, xp)

plt.plot(xp, yhat, '-', label='Ridge Initial Prediction')
plt.plot(xts, yts, 'o', label='Test Data')
plt.xlabel('Scaled Temperature')
plt.ylabel('Scaled Expansion')
plt.title('Prediction with Ridge Initial Fit')
plt.legend()
plt.ylim([-0.5, 1.5])
```

```
(-0.5, 1.5)
```



Creating a Loss Function

We can now use gradient descent to improve our initial estimate. Complete the following function to compute

```
f(w) = 0.5*\sum_i (y[i] - yhat[i])^2
```

and `fgrad`, the gradient of `f(w)`.

```
def feval(w,x,y):

    # TODO 13.  Parse w
    # a = ...
    # b = ...

    d = (len(w)-1)//2
    a = w[:d+1]
    b = w[d+1:]

    # TODO 14.  Znum[i,j] = x[i]**j
    N = len(x)
    Znum = np.zeros((N, d+1))
    for j in range(d+1):
        Znum[:, j] = x**j

    # TODO 15.  Zden[i,j] = x[i]**(j+1)
    Zden = np.zeros((N, d))
    for j in range(d):
        Zden[:, j] = x**(j+1)

    # TODO 16.  Compute yhat
    # Compute the numerator and denominator
    num = Znum.dot(a)
    den = 1 + Zden.dot(b)
    yhat = num / den

    # TODO 17.  Compute loss
    # f = ...
    error = yhat - y
    f = 0.5 * np.sum(error**2)

    # TODO 18.  Compute gradients
    # fgrad = ...
    dl_dyhat = error
    da_grad = np.zeros(d+1)
    for j in range(d+1):
        da_grad[j] = np.sum(dl_dyhat * (x**j) / den)

    db_grad = np.zeros(d)
    for j in range(d):
        db_grad[j] = np.sum(dl_dyhat * (-yhat * (x**(j+1)) / den))

    fgrad = np.concatenate((da_grad, db_grad))
    return f, fgrad
```

Test the gradient function:

- Take `w0=winit` and compute `f0,fgrad0 = feval(w0,xtr,ytr)`
- Take `w1` very close to `w0` and compute `f1,fgrad1 = feval(w1,xtr,ytr)`
- Verify that `f1-f0` is close to the predicted value based on the gradient.

```
# TODO 19
w0 = winit
f0, fgrad0 = feval(w0, xtr, ytr)

step_size = 1e-6
dx = np.random.normal(0, 1, size=w0.shape)
dx = step_size * dx / np.linalg.norm(dx)
w1 = w0 + dx

f1, fgrad1 = feval(w1, xtr, ytr)
df_predicted = np.dot(fgrad0, dx)
df_actual = f1 - f0

print(f"Actual change in loss (f1 - f0): {df_actual:e}")
print(f"Predicted change in loss (fgrad0^T * dx): {df_predicted:e}")
print(f"Relative difference: {abs(df_actual - df_predicted) / abs(df_actual):e}")
```

```
Actual change in loss (f1 - f0): -1.928537e-06
Predicted change in loss (fgrad0^T * dx): -1.928583e-06
Relative difference: 2.374775e-05
```

Implement gradient descent

We will now try to minimize the loss function with gradient descent. Using the function `feval` defined above, implement gradient descent. Run gradient descent with a step size of `alpha=1e-6` starting at `w=winit`. Run it for `nit=10000` iterations. Compute `fgd[it]`= the objective function on iteration `it`. Plot `fgd[it]` vs. `it`.

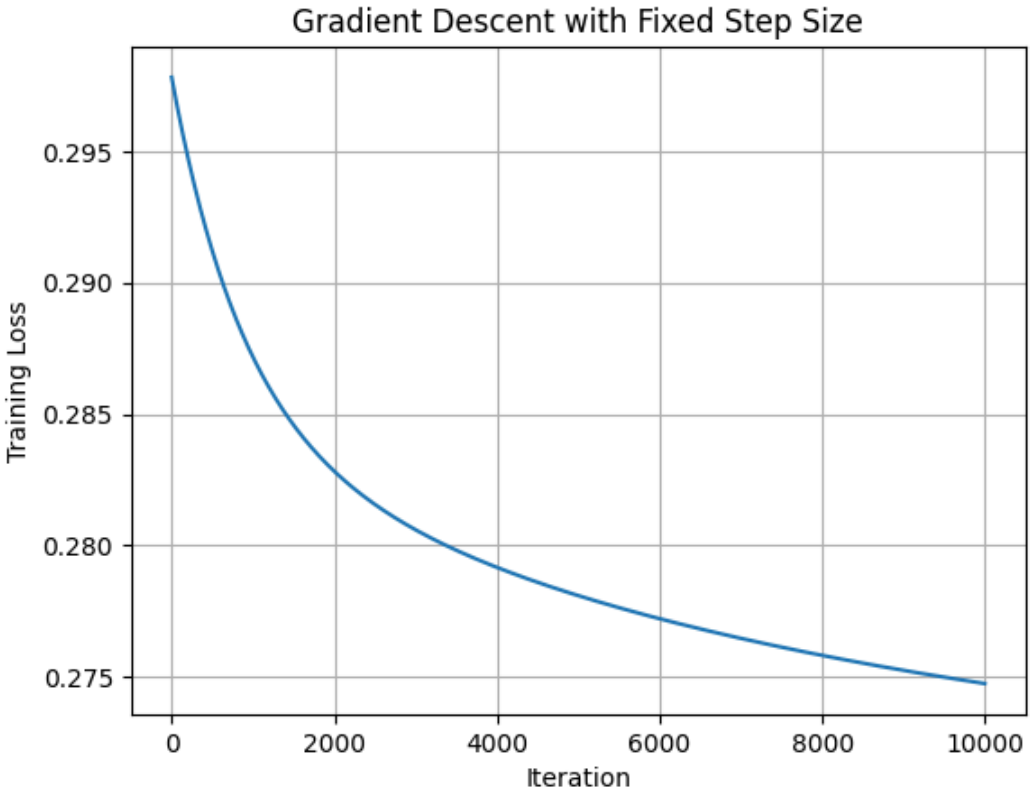
You should see that the training loss decreases, but it still hasn't converged after 10000 iterations.

```
# TODO 20
# fgd = ...
nit = 10000
step = 1e-6

w = winit.copy()
fgd = np.zeros(nit)

for it in range(nit):
    f, fgrad = feval(w, xtr, ytr)
    fgd[it] = f
    w = w - step * fgrad

plt.plot(fgd)
plt.xlabel('Iteration')
plt.ylabel('Training Loss')
plt.title('Gradient Descent with Fixed Step Size')
plt.grid(True)
```



Now, try to get a faster convergence with adaptive step-size using the Armijo rule. Implement the gradient descent with adaptive step size. Let `fadapt[it]` be the loss function on iteration `it`. Plot `fadapt[it]` and `fgd[it]` vs. `it` on the same graph. You should see a slight improvement, but not much.

```
# TODO 21
# fadapt = ...
nit = 10000
step = 1e-6  # Initial step

c = 0.5
w = winit.copy()
fadapt = np.zeros(nit)

f0, fgrad0 = feval(w, xtr, ytr)
for it in range(nit):
    fadapt[it] = f0

    step = step * 1.5

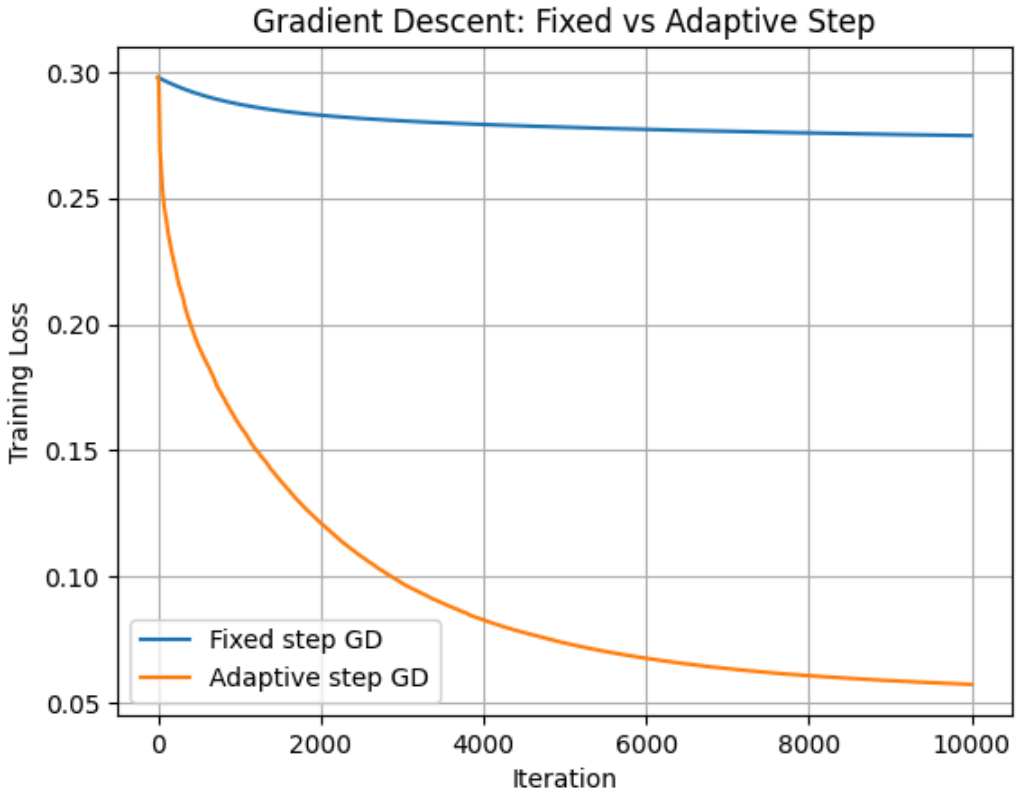
    w1 = w - step * fgrad0
    f1, fgrad1 = feval(w1, xtr, ytr)

    while f1 > f0 - c * step * np.sum(fgrad0**2) and step > 1e-15:
        step = step * 0.5
        w1 = w - step * fgrad0
        f1, fgrad1 = feval(w1, xtr, ytr)

    w = w1
```

```
    f0, fgrad0 = f1, fgrad1

plt.plot(fgd, label='Fixed step GD')
plt.plot(fadapt, label='Adaptive step GD')
plt.xlabel('Iteration')
plt.ylabel('Training Loss')
plt.title('Gradient Descent: Fixed vs Adaptive Step')
plt.legend()
plt.grid(True)
```



Using he final estimate for `w` from the adaptive step-size plot the predicted values of the `yhat` vs. `x` usfor 1000 values `x` in `[0,1]`. On the same plot, plot `yhat` vs. `x` for the initial parameter `w=winit`. Also, plot `yts` vs. `xts`. You should see that gradient descent was able to improve the estimat slightly, although the initial estimate was not too bad.

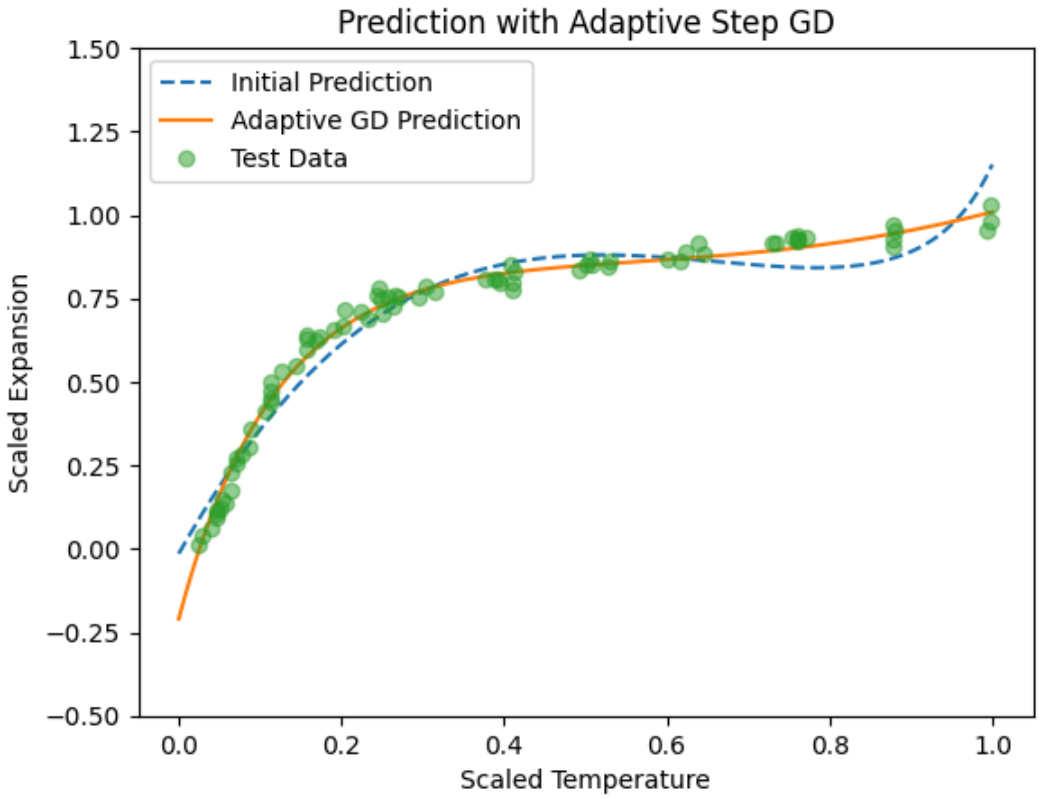
```
# TODO 22
# xp = np.linspace(...)
# yhat = ...
# plot(...)

xp = np.linspace(0, 1, 1000)
yhat_adapt = predict(w, xp)
yhat_init = predict(winit, xp)

plt.plot(xp, yhat_init, '--', label='Initial Prediction')
plt.plot(xp, yhat_adapt, '-', label='Adaptive GD Prediction')
plt.plot(xts, yts, 'o', label='Test Data', alpha=0.5)
plt.xlabel('Scaled Temperature')
plt.ylabel('Scaled Expansion')
```

```
plt.title('Prediction with Adaptive Step GD')
plt.legend()
plt.ylim([-0.5, 1.5])
```

```
(-0.5, 1.5)
```



Momentum Gradient Descent

This section is bonus.

One way to improve gradient descent is to use *momentum*. In momentum gradient descent, the update rule is:

```
f, fgrad = feval(w,...)
z = beta*z + fgrad
w = w - step*z
```

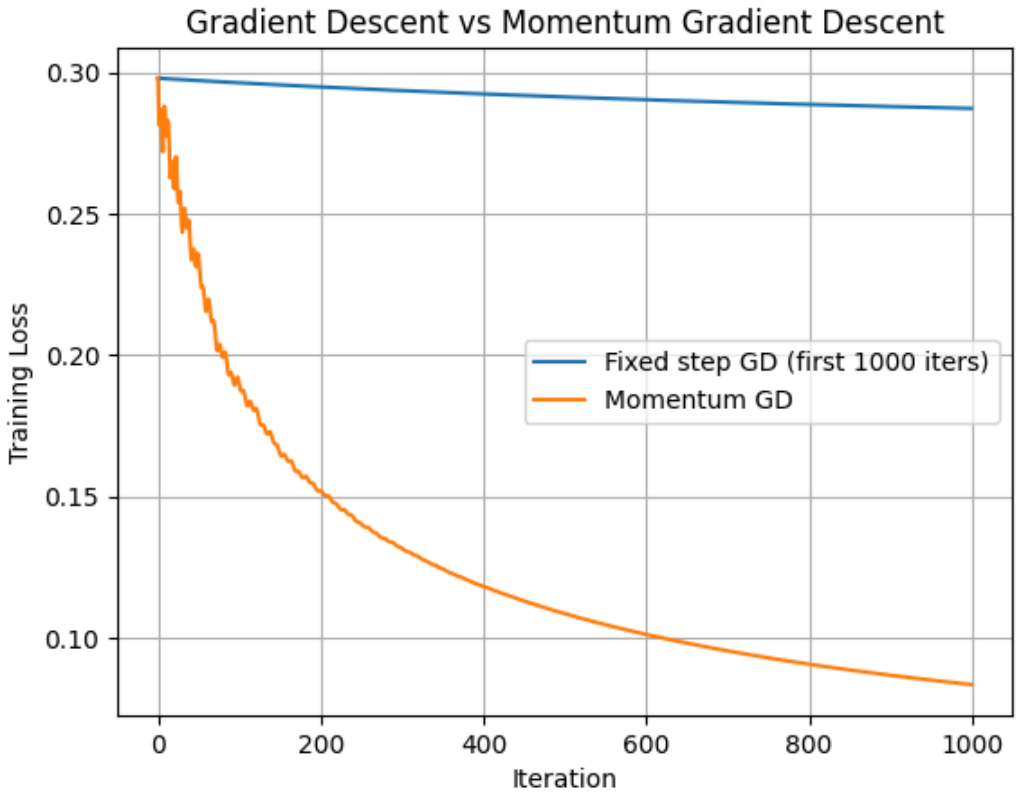
This is similar to gradient descent, except that there is a second order term on the gradient. Implement this algorithm with `beta = 0.99` and `step=1e-3`. Compare the convergence of the loss function with gradient descent.

```
# TODO 23
nit = 1000
step = 1e-3
beta = 0.99

w = winit.copy()
z = np.zeros_like(w)
fmom = np.zeros(nit)
```

```
for it in range(nit):
    f, fgrad = feval(w, xtr, ytr)
    fmom[it] = f
    z = beta * z + fgrad
    w = w - step * z

plt.plot(range(nit), fgd[:nit], label='Fixed step GD (first 1000 iters)')
plt.plot(range(nit), fmom, label='Momentum GD')
plt.xlabel('Iteration')
plt.ylabel('Training Loss')
plt.title('Gradient Descent vs Momentum Gradient Descent')
plt.legend()
plt.grid(True)
```

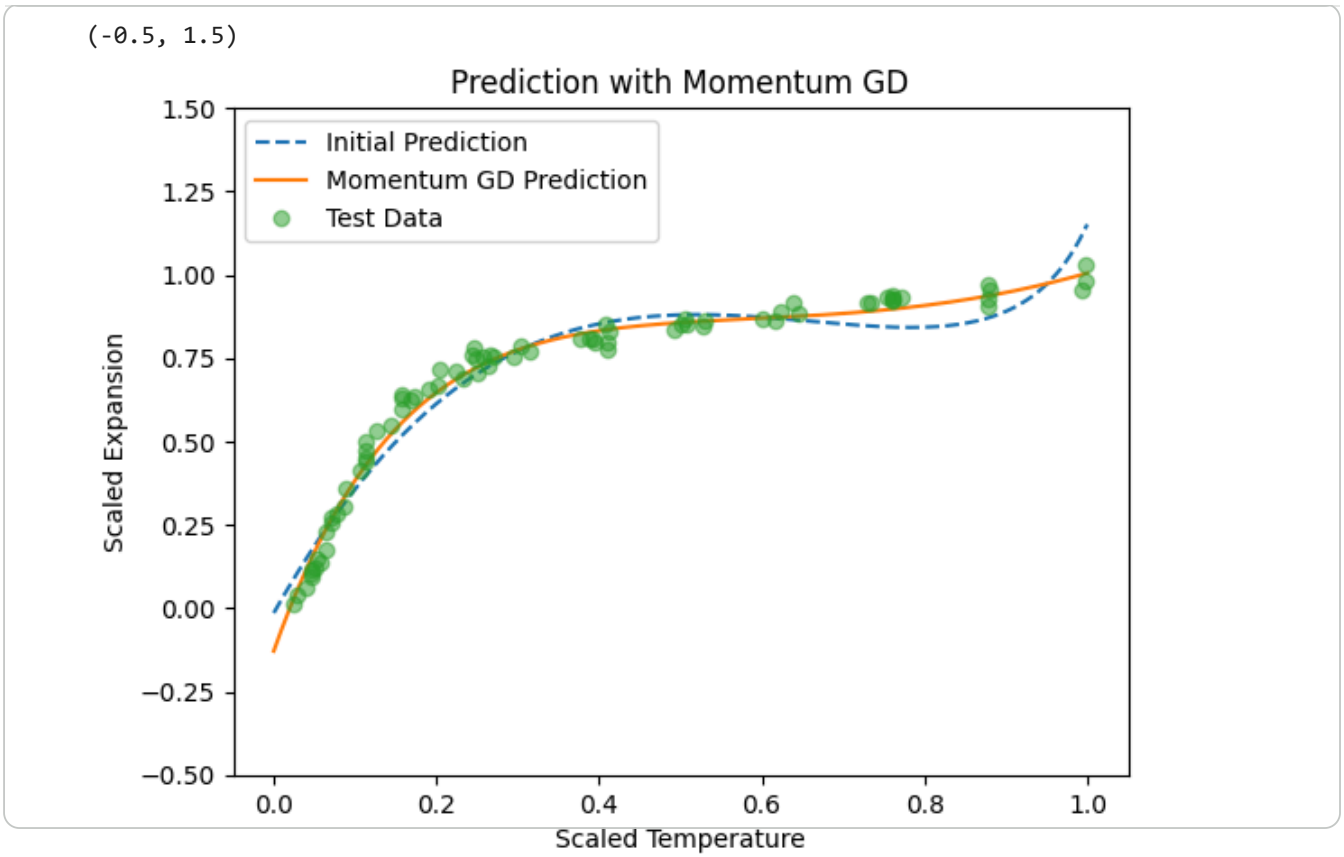



```
# TODO 24
# plot yhat vs. x

xp = np.linspace(0, 1, 1000)
yhat_mom = predict(w, xp)

plt.plot(xp, yhat_init, '--', label='Initial Prediction')
plt.plot(xp, yhat_mom, '-', label='Momentum GD Prediction')
plt.plot(xts, yts, 'o', label='Test Data', alpha=0.5)
plt.xlabel('Scaled Temperature')
plt.ylabel('Scaled Expansion')
plt.title('Prediction with Momentum GD')
plt.legend()
plt.ylim([-0.5, 1.5])
```

```
(-0.5, 1.5)
```



Beyond This Lab